Regression Testing with ReFrame on Toubkal Supercomputer

Mohamed Karim Bezine

June 17, 2025

1 ReFrame

ReFrame is a powerful framework designed for writing and automating system regression tests and benchmarks, specifically targeted at High-Performance Computing (HPC) systems. It provides a structured and efficient approach to executing tests while ensuring portability across different computing environments.

This report covers the following aspects:

- **Installation of ReFrame:** A step-by-step guide on setting up ReFrame in an HPC environment.
- **Regression Test Pipeline in ReFrame:** Understanding the structure of test execution and automation within ReFrame.
- **STREAM and HPL Benchmarks Integration:** Integration of the STREAM and HPL benchmarks in ReFrame.
- **Software Installation Tests Integration:** Integrated checks to verify the proper functioning of the installed software within the HPC cluster environment.
- **Functionality Tests Integration:** Integrated checks to verify that the working environment is correctly configured.
- **Future Work:** Potential enhancements and next steps.

2 ReFrame Installation

The installation process begins by cloning the official ReFrame repository and executing the *bootstrap.sh* script, , as illustrated in Figure 1.pwd:

```
git clone https://github.com/reframe-hpc/reframe.git
cd reframe
./bootstrap.sh
vim ~/.bashrc
```

Figure 1: ReFrame installation commands using bash script.

After that, the necessary environment variable must be set by adding the corresponding lines to the *.bashrc* file, as shown in Figure 2.

```
REFRAME_HOME=<path_to_cloned_repository>
export PATH=$REFRAME_HOME/bin:$PATH
```

Figure 2: Setting Environment Variable

Finally, after sourcing the updated *.bashrc* file, the installation is verified using the *reframe -V* command, as depicted in Figure 3. A successful installation will print the ReFrame version, confirming that the environment is properly configured.

```
source ~/.bashrc
reframe -V
```

Figure 3: Verifying Installation

3 Regression Test Pipeline in ReFrame

The regression test pipeline in ReFrame consists of multiple stages that validate system compatibility, configure the testing environment, execute tests, and analyze results.

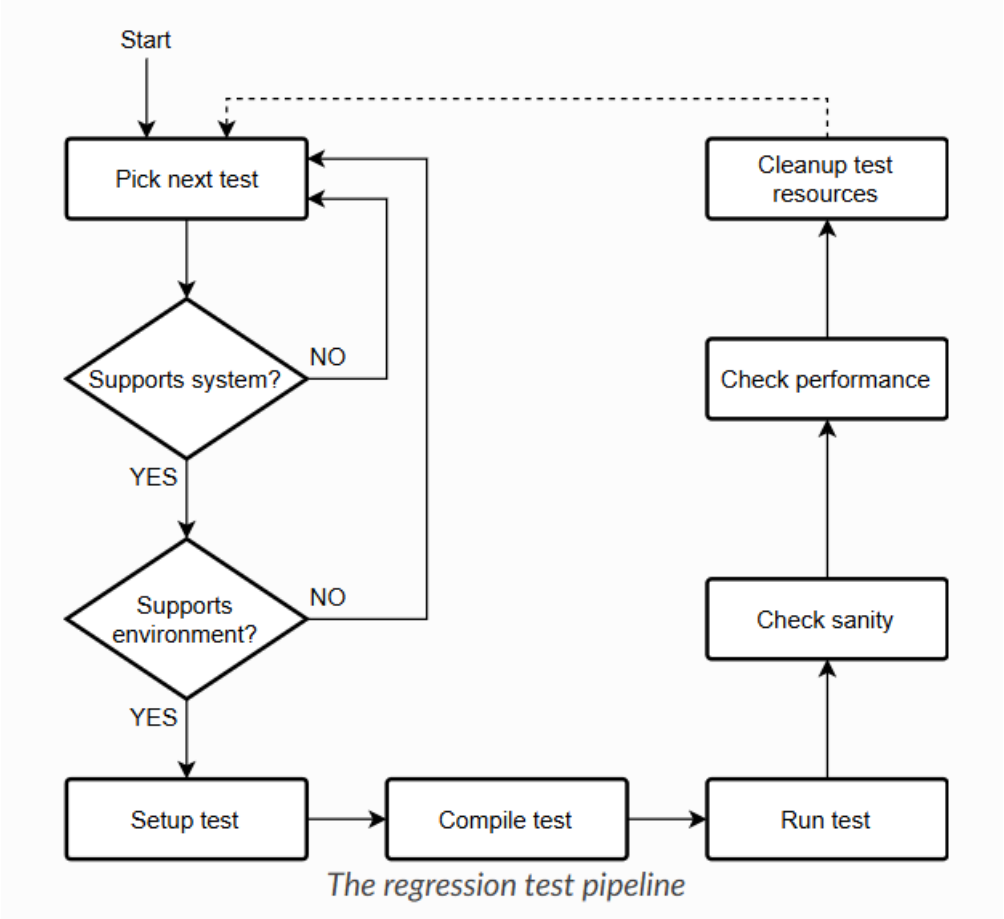


Figure 4: Regression Test Pipeline

Figure 4 illustrates the flow of the regression test pipeline.

- **Test Selection**: The pipeline begins by selecting the next test to execute. This is needed for systematic and automated validation.
- **System Compatibility Check**: Before execution, the test is checked for system compatibility.
  - If the system does not support the test, the process loops back to select another test.
  - If the system supports the test, it proceeds to the next stage.
- **Environment Compatibility Check**: The test must also be compatible with the execution environment.
  - If the environment is not supported, the process loops back to select another test.
  - If the environment is supported, the test setup begins.
- **Test Setup**: The required configurations and dependencies are set up before test execution.
- **Compilation**: If necessary, the test code or scripts are compiled to ensure they are ready for execution.
- **Execution**: The compiled test is executed on the system.
- **Sanity Check**: A preliminary verification is conducted to ensure that the test ran correctly and produced valid results.
- **Performance Analysis**: Performance metrics are checked to determine whether the test meets required benchmarks.
- **Cleanup**: Temporary resources used during the test execution are removed.
- **Loop Back**: After cleanup, the pipeline loops back to pick the next test, ensuring continuous validation.

4 Benchmarks Integration in ReFrame

4.1 Benchmarks Configuration File

```
site_configuration = {
    'systems': [
        {
            'name': 'toubkal_supercomputer',
            'descr': 'Toubkal SuperComputer',
            'modules_system': 'lmod',
            'hostnames': ['.*'],
            'partitions': [
                {
                    'name': 'gpu_h100',
                    'scheduler': 'slurm',
                    'launcher': 'local',
                    'access': [
                        '-A', 'SW_STACK-373LCD9R8IO-DEFAULT-GPU',
                        '-p', 'gpu_h100',
                        '--qos=premium-gpu-temp',
                        '--gres=gpu:1'
                    ],
                    'environs': ['cuda', 'vasp_h100'],
                },
                {
                    'name': 'gpu',
                    'scheduler': 'slurm',
                    'launcher': 'local',
                    'access': [
                        '-A', 'sw_stack-373lcd9r8io-default-gpu',
                        '-p', 'gpu',
                        '--gres=gpu:1'
                    ],
                    'environs': ['cuda', 'vasp_a100'],
                }
            ]
        }
    ],
```

Figure 5: Stream Configuration File (Part 1)

```
    'environments': [
        {
            'name': 'cuda',
            'cc': 'nvcc',
            'cxx': 'nvcc',
            'modules': ['CUDA/12.1.0'],
        },
        {
            'name': 'vasp_h100',
            'modules': ['vasp/6.5.0-gpu_h100'],
        },
        {
            'name': 'vasp_a100',
            'modules': ['vasp/6.5.0-gpu_a100'],
        }
    ]
}
```

Figure 6: Stream Configuration File (Part 2)

The figure 5 and 6 illustrate the Stream Configuration File which covers:

- **System Definition**:
  - Declares the compute system name *toubkal_supercomputer* and a wildcard hostname pattern to match all nodes.
  - Specifies the use of the *lmod* module system to handle software environment management on the cluster.
- **Partition Configuration**:
  - Each partition (e.g., *gpu_h100*, *gpu*) represents a hardware resource queue with:
    - A Slurm job scheduler and a *local* launcher (indicating no MPI execution is required).
    - Specific Slurm options for accessing GPUs, project accounts, and QoS settings.
  - Assigns valid environments (*cuda*, *vasp_h100*, or *vasp_a100*) for each partition, allowing tests to load the appropriate software.
  - Specifies the valid environments for the partition.
  - In this case, an Intel environment is defined for compiling the STREAM benchmark using the Intel compiler.
- **Environment Setup**:
  - Defines named environments with associated modules and compilers.
  - For instance, the *cuda* environment uses the *nvcc* compiler and loads the *CUDA/12.1.0* module.
  - The *vasp_h100* and *vasp_a100* environments load different versions of the VASP software optimized for different GPU architectures.

4.2 HPL Benchmark Reframe Script

```
import reframe as rfm
import reframe.utility.sanity as sn
@rfm.simple_test
class stream_h100(rfm.RunOnlyRegressionTest):
    descr = 'STREAM on H100 GPU'
    valid_systems      = ['toubkal_supercomputer:gpu_h100']
    valid_prog_environs= ['cuda']
    num_tasks          = 1
    num_gpus_per_node  = 1
    time_limit         = '10m'
    exclusive_access   = True
    # Paths to your container and data
    sif_path  = <path_to_your_sif_file>
    dat_mount = <path_to_your_directory_of_your_dat_file>
    executable      = 'bash'
    executable_opts = []
    @run_before('run')
    def set_executable_opts(self):
        self.executable_opts = ['-c', f'''
            echo "=== Checking GPU ==="
            nvidia-smi
            echo "=== Checking Singularity Bind ==="
            singularity exec --bind {self.dat_mount}:/mnt {self.
               ↪ sif_path} \
              ls -l /mnt/HPL.dat
            echo "=== Running STREAM Benchmark ==="
            singularity run --nv {self.sif_path} \
              /workspace/stream-gpu-linux-x86_64/stream-gpu-test.
                 ↪ sh --n 1300000000 \
              --dat /mnt/hpcg.dat \
              > STREAM.out 2>&1

            cat STREAM.out
        '''']
    @run_before('setup')
    def do_not_stage_container(self):
        self.readonly_files = []
    @sanity_function
    def validate_gpu_and_hpl(self):
        return sn.all([
        sn.assert_found(r'Copy', self.stdout),
        sn.assert_found(r'Scale', self.stdout),
        sn.assert_found(r'Add', self.stdout),
        sn.assert_found(r'Triad', self.stdout)
        ])
```

Figure 7: Stream Test Execution File

NB: To execute the same test on an A100 GPU, simply change the partition name in *valid_systems* (e.g., to *toubkal_supercomputer:gpu*) and update the description accordingly to reflect the target hardware.

The code presented in Figure 7 defines a ReFrame test to execute the STREAM benchmark inside a Singularity container on an H100 GPU node. Below is a detailed breakdown of the test configuration:

- **@rfm.simple_test**: Declares the class as a ReFrame test using the *RunOnlyRegressionTest* base class, suitable for tests that do not require compilation.
- **valid_systems**: Restricts execution to the *gpu_h100* partition of the *toubkal_supercomputer*, ensuring compatibility with H100 GPUs.
- **valid_prog_environs**: Specifies that the test should run in a programming environment that includes CUDA support.
- **num_tasks** and **num_gpus_per_node**: Indicates the test will run with a single task and use one GPU, aligning with GPU-bound workloads like STREAM.
- **exclusive_access**: Requests exclusive use of the node to eliminate resource contention during the benchmark.
- **sif_path** and **dat_mount**: Define the paths to the Singularity container file and the directory to bind into the container (which holds input files like *HPL.dat*).
- **set_executable_opts**: This method sets the executable options that include:
  - A GPU diagnostic command using *nvidia-smi*
  - A check to confirm the bind mount by listing files inside the container
  - The STREAM benchmark execution using *singularity run –nv*
  - Logging of output into *STREAM.out*
- **do_not_stage_container**: Overrides default ReFrame behavior to avoid staging the container file. This is useful when using large pre-deployed containers already accessible on shared filesystems.
- **validate_gpu_and_hpl**: The *sanity_function* that validates successful benchmark execution by checking the presence of all four STREAM operations: *Copy*, *Scale*, *Add*, and *Triad* in the output.

This setup ensures a reliable and modular approach for running memory bandwidth benchmarks across modern GPU clusters with containerized environments.

4.3 Stream Benchmark Reframe Script

```
import reframe as rfm
import reframe.utility.sanity as sn
@rfm.simple_test
class hpl_h100(rfm.RunOnlyRegressionTest):
    descr = 'HPL on H100 GPU'
    valid_systems = ['toubkal_supercomputer:gpu_h100']
    valid_prog_environs = ['cuda']
    num_tasks = 1
    num_gpus_per_node = 1
    time_limit = '10m'
    exclusive_access = True
    sif_path = <path_to_your_sif_file>
    dat_mount = <path_to_your_directory_of_your_dat_file>
    executable = 'bash'
    executable_opts = []
    @run_before('run')
    def set_executable_opts(self):
        self.executable_opts = ['-c', f'''
            echo "=== Checking GPU ==="
            nvidia-smi
            echo "=== Checking Singularity Bind ==="
            singularity exec --bind {self.dat_mount}:/mnt {self.
               ↪ sif_path} \
              ls -l /mnt/HPL.dat
            echo "=== Running HPL Benchmark ==="
            singularity exec --nv --bind {self.dat_mount}:/mnt {
               ↪ self.sif_path} \
              mpirun -np 1 /workspace/hpl-linux-x86_64/hpl.sh \
              --dat /mnt/HPL.dat \
              --gpu-affinity 0 \
              --no-multinode \
              --cuda-compat > HPL.out 2>&1
            echo "=== Output ==="
            cat HPL.out
        '''']
    @run_before('setup')
    def do_not_stage_container(self):
        self.readonly_files = []
    @run_before('sanity')
    def set_sanity_patterns(self):
        self.sanity_patterns = sn.all([
            sn.assert_found(r'End of Tests\.', 'HPL.out'),
            sn.assert_found(r'0 tests completed and failed
               ↪ residual checks', 'HPL.out'),
            sn.assert_found(r'0 tests skipped because of illegal
               ↪ input values\.', 'HPL.out')
        ])
```

Figure 8: HPL Test Execution File

Before running the benchmark, the script:

- Displays GPU status using *nvidia-smi*.
- Verifies the presence of the benchmark input file *HPL.dat* inside the container.
- Runs the HPL benchmark with MPI and CUDA support, capturing output in *HPL.out*.

Sanity checks ensure:

- The benchmark completes successfully (checks for *End of Tests.*).
- No residual check failures occurred.
- No tests were skipped due to illegal input values.

NB: To execute the same test on an A100 GPU, simply change the partition name in *valid_systems* (e.g., to *toubkal_supercomputer:gpu*) and update the description accordingly to reflect the target hardware.

5 Testing Installed Software on the Cluster

5.1 KNN CUDA Script

```
import reframe as rfm
import reframe.utility.sanity as sn
@rfm.simple_test
class knn_cuda_h100(rfm.RunOnlyRegressionTest):
    valid_systems = ['toubkal_supercomputer:gpu_h100']
    valid_prog_environs = ['*']
    num_tasks = 1
    num_tasks_per_node = 1
    executable = 'singularity'
    executable_opts = [
        'exec',
        '--nv',
        '-B <path_to_your_shared_directory_to_bind>:/mnt',
        '<path_to_your_directory_of_your_dat_file>',
        'python3.10', '/mnt/tests/test.py'
    ]
    @sanity_function
    def validate_job_done(self):
        return sn.assert_found(r'KNN CUDA is working fine on GPU!
            ↪ ', self.stdout)
```

Figure 9: ReFrame test for validating KNN CUDA execution

This script runs a Python-based KNN benchmark using CUDA inside a Singularity container.

- The container is mounted with a shared directory containing the benchmark script.
- The test is executed via ‘python3.10‘ inside the container with GPU support (‘–nv‘).
- The sanity function checks for a success message in the output: *KNN CUDA is working fine on GPU!*.

5.2 Quantum ESPRESSO Script

```
import reframe as rfm
import reframe.utility.sanity as sn
@rfm.simple_test
class qe_h100(rfm.RunOnlyRegressionTest):
    valid_systems = ['toubkal_supercomputer:gpu_h100']
    valid_prog_environs = ['cuda']
    num_tasks = 1
    num_tasks_per_node = 1
    executable = 'singularity'
    executable_opts = [
        'exec',
        '--nv',
        '-B <path_to_your_espresso_directory>:/host_pwd',
        '--pwd', '/host_pwd',
        '<path_to_your_sif_file>',
        'mpirun', '-np', '1', 'pw.x', '-in', '/host_pwd/quantum/
          ↪ input.in'
    ]
    input_file = 'input.in'
    sourcesdir = '<path_to_your_espresso_directory>'
    @sanity_function
    def validate_job_done(self):
        return sn.assert_found(r'JOB DONE', self.stdout)
```

Figure 10: ReFrame test for running Quantum ESPRESSO simulation.

This script launches a Quantum ESPRESSO simulation in a containerized environment using Singularity.

- The container is mounted with the simulation input directory and the working directory is set inside (‘–pwd‘).
- The executable *pw.x* (Plane-Wave Self-Consistent Field) is launched using MPI.
- Sanity check verifies that the job completed successfully by searching for the string *JOB DONE*.

5.3 VASP Script

```
import reframe as rfm
import reframe.utility.sanity as sn
@rfm.simple_test
class vasp_h100(rfm.RunOnlyRegressionTest):
    valid_systems = ['toubkal_supercomputer:gpu_h100']
    valid_prog_environs = ['vasp_h100']
    sourcesdir = '<path_to_your_vasp_test_directory>'
    num_tasks = 1
    num_tasks_per_node = 1
    num_cpus_per_task = 1
    exclusive_access = True
    use_multithreading = False
    executable = 'mpirun'
    executable_opts = ['-np', '1', 'vasp_std']
    @sanity_function
    def validate_vasp_run(self):
        return sn.assert_found(r'reached required accuracy', self
            ↪ .stdout)
```

Figure 11: ReFrame test for VASP execution on H100 GPU.

This test runs the VASP (Vienna Ab initio Simulation Package) code using ReFrame.

- The working directory is set to a location containing the VASP input files.
- VASP is executed with *mpirun -np 1 vasp_std*.
- The test ensures convergence by verifying that the required accuracy was reached.

NB: To run the same test on a different GPU partition (e.g., A100), you must update the *valid_systems* field in the script accordingly (e.g., replace *toubkal_supercomputer:gpu_h100* with *toubkal_supercomputer:gpu*). Additionally, ensure that the corresponding environment (e.g., *vasp_a100*) and compatible executable are available on that partition. All partitions are available in the configuration file.

6 Functionality Tests

6.1 Environment Check

The environment check, involves running a series of system diagnostic commands to verify the key components of the system environment, such as the hostname, Slurm version, kernel version, and hardware details. The check is done through a shell script executed by a ReFrame test.

```
#!/bin/bash
echo "========== System Info =========="
echo "Hostname:"
hostname

echo -e "\nSlurm Version:"
sinfo -V

echo -e "\nNodes in partition 'gpu':"
sinfo -p gpu -N

echo -e "\nNodes in partition 'gpu_h100':"
sinfo -p gpu_h100 -N

echo -e "\nOperating system version:"
cat /etc/redhat-release

echo -e "\nKernel version:"
uname -r

echo -e "\nCPU and HT/SMT:"
lscpu

echo -e "\nRAM:"
free -m

echo -e "\nMellanox OFED:"
ofed_info -n 2>/dev/null || echo "OFED not found"

echo -e "\nIB adapters and firmware:"
ibstat 2>/dev/null || echo "IB not found"

echo -e "\nLocal storage:"
lsblk
df -h

echo -e "\nLustre version:"
cat /sys/fs/lustre/version 2>/dev/null || echo "Lustre not found"
```

Figure 12: System diagnostics script – Part 1: CPU, OS, and Storage Info

```
# GPU-specific diagnostics
if [[ $HOSTNAME == *gpu* || $CUDA_VISIBLE_DEVICES ]]; then
    echo -e "\n========== GPU Diagnostics =========="
    echo -e "\nNVIDIA driver/kernel module:"
    nvidia-smi || echo "nvidia-smi not available"
    echo -e "\nCUDA Version:"
    cuda_version=$(nvidia-smi | grep "CUDA Version" | sed -E 's
       ↪ /.*CUDA Version: ([0-9]+\.[0-9]+).*/\1/')
    if [ -z "$cuda_version" ]; then
        echo "CUDA Version not found!"
    else
        echo "$cuda_version"
    fi
    echo -e "\nGPU Inventory:"
    nvidia-smi --query-gpu=index,name,driver_version,memory.total
       ↪ ,memory.used --format=csv,noheader
    echo -e "\nGPU Count by Type:"
    echo "H100 count: $(nvidia-smi --query-gpu=name --format=csv,
       ↪ noheader | grep -i h100 | wc -l)"
    echo "A100 count: $(nvidia-smi --query-gpu=name --format=csv,
       ↪ noheader | grep -i a100 | wc -l)"

    echo -e "\nGPU RAM Info:"
    nvidia-smi --query-gpu=name,memory.total --format=csv,
       ↪ noheader \
        | awk -F',' '{gsub(/ MiB/, "", $2); printf "GPU: %s | RAM
           ↪ : %s MiB\n", $1, $2}'
    echo -e "\nTensor Core Support (Ampere/Hopper):"
    nvidia-smi --query-gpu=index,name,compute_cap --format=csv,
       ↪ noheader
fi
echo -e "\n========== SLURM Job Info =========="
echo "Job ID: $SLURM_JOB_ID"
scontrol show job $SLURM_JOB_ID 2>/dev/null || echo "No job info
   ↪ available"
```

Figure 13: System diagnostics script – Part 2: GPU and SLURM Job Info

Figures 12 and 13 present a two-part diagnostic script. The first part gathers CPU, memory, storage, and OS-related system information. The second part focuses on GPU diagnostics and SLURM job environment verification.

```
import reframe as rfm
import reframe.utility.sanity as sn
@rfm.simple_test
class env_check(rfm.RunOnlyRegressionTest):
    def __init__(self):
        super().__init__()
        self.executable = '<path_to_your_environement_script>'
        self.executable_opts = []
        self.valid_systems = ['*']
        self.valid_prog_environs = ['*']
        self.time_limit = '1m'
        self.num_tasks = 1
        self.num_tasks_per_node = 1
        self.num_cpus_per_task = 1
        self.pre_run = ['chmod +x env.sh']
    @sanity_function
    def validate(self):
        common_checks = [
            sn.assert_found(r'Hostname:', self.stdout),
            sn.assert_found(r'Slurm Version:', self.stdout),
            sn.assert_found(r"Nodes in partition 'gpu':", self.
               ↪ stdout),
            sn.assert_found(r"Nodes in partition 'gpu_h100':",
               ↪ self.stdout),
            sn.assert_found(r'Operating system version:', self.
               ↪ stdout),
            sn.assert_found(r'Kernel version:', self.stdout),
            sn.assert_found(r'CPU and HT/SMT:', self.stdout),
            sn.assert_found(r'RAM:', self.stdout),
            sn.assert_found(r'Mellanox OFED:', self.stdout),
            sn.assert_found(r'IB adapters and firmware:', self.
               ↪ stdout),
            sn.assert_found(r'Local storage:', self.stdout),
            sn.assert_found(r'Lustre version:', self.stdout),
            sn.assert_found(r'========== SLURM Job Info
               ↪ ==========', self.stdout),
        ]
        hostname_check = sn.extractsingle(r'Hostname:\s*(\S+)',
           ↪ self.stdout, 1)
        gpu_checks = sn.evaluate(hostname_check)
        if 'gpu' in gpu_checks.lower():
            common_checks += [
                sn.assert_found(r'CUDA Version:', self.stdout),
                sn.assert_found(r'GPU Inventory:', self.stdout),
                sn.assert_found(r'GPU Count by Type:', self.
                   ↪ stdout),
                sn.assert_found(r'Tensor Core Support \(Ampere/
                   ↪ Hopper\):', self.stdout),
            ]
        return sn.all(common_checks)
```

Figure 14: ReFrame test for VASP execution on H100 GPU

The Figure 14 shows the regression test that runs the *Environment Check Bash Script* to check system environment details. The test validates that specific patterns (such as Slurm, Kernel, CPU, RAM, Mellanox OFED, and storage) appear in the script's output. Additionally, if the detected hostname indicates a GPU node, the test performs extended GPU-specific checks. These include the verification of the CUDA version, GPU inventory, GPU count by type (e.g., H100, A100), and tensor core support (Ampere/Hopper). This ensures that both CPU and GPU environments are correctly set up and ready for use.

6.2 Slurm Check

```
import reframe as rfm
import reframe.utility.sanity as sn
import getpass
@rfm.simple_test
class SlurmCheckTest(rfm.RunOnlyRegressionTest):
    variant = parameter(['slurmctld', 'squeue', 'sacct'])
    descr = "Reframe Slurm Check Test"
    valid_systems = ['*']
    valid_prog_environs = ['*']
    @run_before('run')
    def setting_variables(self):
        if self.variant == "slurmctld":
            self.executable = 'scontrol ping'
            self.sanity_patterns = sn.assert_found(
                r'Slurmctld\(primary\) at .* is UP',
                self.stdout
            )
        elif self.variant == "squeue":
            self.executable = f'squeue -u {getpass.getuser()}'
            self.sanity_patterns = sn.assert_found(r'NODELIST',
              ↪ self.stdout)
        elif self.variant == "sacct":
            self.executable = 'sacct -u {}'.format(getpass.
              ↪ getuser())
            self.sanity_patterns = sn.assert_found(r'JobID\s+
              ↪ JobName\s+Partition', self.stdout)
```

Figure 15: Slurm Check Test

The *SlurmCheckTest* is a parameterized ReFrame test designed to verify the health and availability of key SLURM services on the cluster. Depending on the selected variant (*slurmctld*, *squeue*, or *sacct*), the test runs a corresponding SLURM command:

- **slurmctld**: Checks the availability of the SLURM controller using *scontrol ping*, expecting a response indicating the controller is UP.
- **squeue**: Lists the user's currently queued or running jobs using *squeue*, verifying that the standard job table header (*NODELIST*) appears.

- **sacct**: Queries job accounting information with *sacct*, validating the presence of expected headers like *JobID*, *JobName*, and *Partition*.

This test ensures that the core SLURM scheduling components are functional and responsive, which is critical for reliable job submission and tracking on the HPC system.

7 Conclusion

This report presents a comprehensive and automated testing framework designed to verify the proper configuration and functionality of the HPC cluster environment. For future work, we will integrate ReFrame tests into a CI/CD pipeline to ensure continuous automated verification and improve the reliability and efficiency of system maintenance.

References

[1] CSCS Swiss National Supercomputing Centre *Welcome to ReFrame*. Available from: `https://reframe-hpc.readthedocs.io/en/stable/index.html#`

[2] CSCS - Swiss National Supercomputing Centre. *ReFrame: A Regression Testing Framework for HPC Systems*. Available from: `https://github.com/reframe-hpc/reframe`